COURSEWORK SPECIFICATION
COM2016B – Programming with Python
Module Leader: Milto Miltiadou
Academic Year: 2025/26

Title: **Implement a command line version of the board game "Cluedo"**

Submission deadline: **March 23, 2026**

This assessment contributes **70%** of the total module mark and assesses the following **intended learning outcomes**:

1. Implement software to solve a given problem that uses the control structures and data structures available in the Python language.
2. Use industry-standard approaches for testing and developing software
3. Formulate a software design to a problem from a given field or industry beyond computer science or data science.

This is an individual assessment, and you are reminded of the University's regulations on collaboration and plagiarism. You must avoid plagiarism, collusion, and any academic misconduct behaviours. Further details about academic honesty and plagiarism can be found at https://ele.exeter.ac.uk/course/view.php?id=1957.

The University of Exeter is committed to the ethical and responsible use of Generative AI (GenAI) tools in teaching and learning, in line with our academic integrity policies where the direct copying of AI-generated content is included under plagiarism, misrepresentation and contract cheating under definitions and offences in TQA Manual Chapter 12.3. To support students in their use of GenAI tools as part of their assessments, we have developed a category tool that enables staff to identify where use of Gen AI is integrated, assisted, minimal or prohibited in each assessment. This assessment falls under the category of **AI-assisted.** You may use GenAI tools ethically and responsibly to assist in the development of this assessment. In AI-assisted assessments, you must include an GenAI statement (see page 2) in your report and you must include prompts and, where possible, hyperlinks to their output as part of your code submitted. All AI-generated code and prompts must be included at the end of each related .py file as comments. At a potential invited viva, you should be able to demonstrate how you produced AI-generated code and how you built on this content to ensure the work is original.

Declaration of use AI-assisted coursework:

Include this declaration at the start of your work.

Select all that apply with an [X]:

-----✂----- Copy and fill in this declaration at the start of your work -----✂-----

I acknowledge the following uses of GenAI tools in this assessment:

- [ ] I have used GenAI tools to:
    - [ ] develop ideas.
    - [ ] assist with research or gathering information.
    - [ ] help me understand key theories and concepts.
    - [ ] identify trends and themes as part of my data analysis
    - [ ] suggest a plan or structure for my assessment.
    - [ ] give me feedback on a draft.
    - [ ] generate images, figures or diagrams.
    - [ ] proofread and correct grammar or spelling errors.
    - [ ] generate citations or references.
    - [ ] Other: [please specify]
- [ ] I have not used any GenAI tools in preparing this assessment.

I declare that I have referenced all use of GenAI outputs within my assessment in line with the University referencing guidelines.

I certify that all material in this dissertation which is not my own has been identified.

-----✂--------- End of declaration – Do not include 'scissor' lines ---------✂-----

Instructions

You are required to design and implement a command-line version of the board game *Cluedo* using Python. The instructions for the game can be found at the following:

- Video Tutorial: https://www.youtube.com/watch?v=43GvEYJ6vh0 and
- Official Game instructions in pdf: https://www.hasbro.com/common/documents/dad2885d1c4311ddbd0b0800200c9a66/2BFAEC9E5056900B102C3859E9AC6332.pdf

The game should include all essential interactions, such as between users and cards (if applicable). The focus should be on demonstrating core functionality, interactions, and object-oriented design. Game customizations are allowed (e.g., creating a smaller board to make the game finish faster), provided they are clearly explained in the dedicated section of the report.

Stage 1 Tasks*:

- Create a Github repository about the project and push changes regularly to demonstrate your progress.
- Use Object Oriented Programming to structure the project using multiple classes (e.g., classes for players, board, resources, property, dice, as required by each game).
- Implement a game loop:
  - Implement a command-line interface to handle user input for gameplay.
  - At the beginning, the game shall ask for number of players, and then request names for each user.
  - Ensure the game progresses according to the player's choices and its rules
  - The game must terminate according to a game-ending condition and declare a winner, if applicable.
  - If the player presses the button *p*, the game must display a summary of the current state of the game.
- Create a requirements.txt file that enables code transferability
- Create a 10 mins video tutorial and demonstration using Panopto:
  - Explain the options the user has at each stage
  - Demonstrate a full game loop (you may edit the video)
  - Show the termination part of the game
  - Your face recording must be included in the video recording

Stage 2 Tasks*:

- Use *pytest* to implement unit tests covering the project's core methods and expected behaviours. To complete this task, you are expected to create a test file for .py file (usually containing a single class definition).
- Implement appropriate code documentation using *sphinx*. The expected output is a folder named doc that, under the *docs\build\html* directory, contains a set of *.html* files.
- Implement error handling to prevent crashes (e.g., invalid user input, out-of-bounds moves).

* Please note that it will take several weeks to cover all Object-Oriented Programming (OOP) content. Unit testing and documentation will be taught later, so please be patient.

! Do NOT do the following:

- Do NOT include a Computer User controlled using AI or random decision making
- Do NOT exceed the 10mins of video. Any seconds after the time limit will be ignored.
- Do NOT record part of the screen. Ensure that typing player's input is visible along with the execution.

Deliverables

You must submit the following into a single .zip file containing the following:

1. **A folder named "code" that includes:**

    - A file named *main.py* file that initiates the start the game
    - Multiple python *.py* files – it is recommended to use one for each developed class
    - A *requirements.txt* listing all required dependencies
    - Code documentation inside the "*doc*" folder (if applicable)
    - Unit Tests developed using Pytest (if applicable)

    Do NOT include the folder containing your environment variables (e.g., myenv).

    Do NOT create subfolders. All .py files, including unit tests, shall be listed inside the folder named "code".

    GenAI Note: You must keep a record of the tools, prompts and outputs used at the end of each .py file. At a potential invited viva, you should be able to demonstrate how you built on the GenAI content to ensure the work is original.

2. **A concise report that contains the following:**

    - The link to your Github Repository. Ensure that the repository is private and invite the Module Leader (https://github.com/Art-n-MathS) as a collaborator.
    - A link to your Video Demonstration/Tutorial, stored on your university OneDrive. The Video must be less than 10 mins and show live instructions on how to play the game, a wide range of game functionalities in practise, and how the game terminates. Ensure that you have updated the access setting to ensure that the Module Leader (m.miltiadou@exeter.ac.uk) and the 2nd marker (Zhixuan Yang - Z.Yang2@exeter.ac.uk) have access to it.
    - A list of tentative adjustments made to speed up testing and termination. If no modification made, please write "no game modifications applied".
    - Class diagrams using Unified Modified Language
    - A list of up to 5 biggest challenges faced and how they were addressed - use only two sentences for each challenge; one to describe the challenge and one to explain what methods/approach you used to solve it.

- A list of up to 5 OOP choices (e.g., inheritance). For each one of them, explain why they were chosen and how they contribute to the efficiency of the implementation - use up to three sentences for each OOP design choice.

The report should be concise and include only the requested information.

	Features Implemented (35%)	Object Oriented Implementation Design (35%)	Error Handling, Documentation and Unit Testing (20%)	Style and Presentation (10%)
Fail (0-39%)	- It is not clear how to play the game. - The game doesn't compile or produces errors. - Unexpected behaviour occurs.	- The game is procedural, and no classes implemented. - The game is within a single .py file. - No list of OOP features in the report or explanation is missing.	- The game crashes when e.g., wrong options are provided. - The documentation is absent. - No Unit Tests included and/or Pytest massively fails once run.	- The instruction provided on how to play the game are not clear. - The code is hard to read, and naming is not meaningful. - Github repository and *requirements.txt* are not provided.
Third (40-49%)	- Basic game loop exists. - Minimal functionalities implemented. - List of challenges faced not included; they may not be challenging enough, or explanation may not clear.	- A couple of classes implemented. - No intermediate or advance OOP demonstrated in your report (e.g., inheritance). - Classes grouped in one file, making code hard to read. - OOP feature list missing or unjustified.	- Minimal error handling; game unstable. - Docstrings present but no HTML documentation. - Very few unit tests; Pytest fails some tests.	- Instructions vague. - Code partially readable; inconsistent name. - Report/video present but poorly structured or contains irrelevant information.
2:2 (50-59%)	- Main functionalities implemented; game runs as expected and termination is demonstrated. - A good list of challenges included, and adequate explanation is provided.	- Several classes implemented. Good UML diagram included. - Some intermediate features (e.g., protected) exist in the OOP design choices list. - The resonate behind the design choices may not be clear or contribute very little to the efficiency of the implementation.	- Error handling works in most cases. - HTML documentation exists but incomplete (missing links/classes). - A few basic unit tests included.	- Folder structure partially followed. - The code is clear, but there is room for improvement e.g., it may include repetitions. - Report understandable but too long.

2:1 (60-69%)	- Game loop works correctly; winner announced. - Wide range of functionalities implemented. - The list of challenges includes and vey good explanation on how they were tackled is provided.	- Multiple classes with good structure, depicted correctly in the UML diagram. - Variety of OOP design features included. - Report justifies OPP usage. Omissions minimal.	- Error Handling robust - HTML documentation complete, links work. Any omissions are minor. - Wide range of unit tests; some advance features for code efficiency are included.	- Clear instructions in report and video. - Code is clear and easy to follow, e.g., meaningful names. Minor improvements may be needed. - Report and video presentation concise and clear.
1st (70-79%)	- Game fully functional with polished features. - Wide range of functionalities implemented. - The list of challenges included and the solutions provided are optimal.	- Well-structured classes with advanced features depicted in the UML diagram. - Variety of OOP design features implemented. - Excellent justification of design choices that demonstrate code efficiency, reproducibility, adaptivity and maintenance.	- Extensive error handling. - HTML documentation complete and well-formatted and all classes are included. - Comprehensive unit tests; includes efficiency improvements.	- Clear, professional instructions are provided in both the video recording and the command-line messages printed in the terminal, guiding players on how to play the game. - Code highly readable and well-organised. - Report is concise and well-structured with no mistakes.
1st (80-100%)	- Game exceeds requirements with innovate features. - An excellent range of functionalities implemented. - Challenges are solved in the most optimal way to enhance code efficiency.	- Outstanding OOP design with advanced patterns and modularity. - Report demonstrates deep understanding of design implementation choices. - Code is efficient, easy to adapt, extend and maintain.	- Extensive error handling with exceptional stability - HTML documentation flawless. - Unit tests thoroughly test the game functionalities with an efficient implementation.	- Outstanding clarity and professionalism in all deliverables. - Code follows best practices rigorously. - Report and video are polished and engaging.